

Day 8 学习总结 - Python 连接 MySQL

学习日期：2026-06-30

核心内容：pymysql 连接数据库、执行 SQL、pandas 与 MySQL 互导数据

一、核心概念

1. pymysql 是什么

`pymysql` 是 Python 和 MySQL 之间的"桥梁", 让 Python 程序可以连接数据库、执行 SQL 语句。

类比 C 语言：

C 语言	Python
<code>#include <mysql.h></code>	<code>import pymysql</code>
<code>mysql_connect(...)</code>	<code>pymysql.connect(...)</code>
<code>mysql_query(...)</code>	<code>cursor.execute(...)</code>

2. 连接 MySQL 的核心流程

步骤	pymysql	类比文件操作
1. 建连接	<code>conn = pymysql.connect(...)</code>	<code>fp = fopen(...)</code>

2. 创建游标	<code>cursor = conn.cursor()</code>	用 fp 直接操作
3. 执行 SQL	<code>cursor.execute(SQL)</code>	<code>fprintf(fp, ...)</code>
4. 读取结果	<code>cursor.fetchall()</code>	<code>fgets(fp, ...)</code>
5. 提交事务	<code>conn.commit()</code>	<code>fflush(fp)</code>
6. 关闭	<code>cursor.close() + conn.close()</code>	<code>fclose(fp)</code>

二、命令用法与完整语法

1. pymysql.connect() - 建立连接

```
conn = pymysql.connect(
    host="localhost",      # MySQL 服务器地址
    user="root",           # 用户名
    password="密码",       # 密码
    database="数据库名",   # 选择数据库
    charset="utf8"         # 字符编码，防止中文乱码
)
```

2. cursor.execute() - 执行 SQL

```
cursor = conn.cursor()    # 创建游标
cursor.execute("SQL语句") # 执行 SQL
result = cursor.fetchall() # 获取所有结果（返回列表，每行是元组）
```

重要：INSERT / UPDATE / DELETE 操作后必须调用 `conn.commit()`，否则数据不会真正写入数据库！SELECT 不需要 commit。

3. pd.read_sql() - 读取数据到 DataFrame

```
import pandas as pd

df = pd.read_sql("SELECT * FROM 表名", conn)
# 直接返回 DataFrame, 比 fetchall() 返回元组更好用
```

参数	说明
sql	字符串, 要执行的 SQL 语句
con	数据库连接对象 (pymysql 的 conn 或 SQLAlchemy 的 engine)

4. df.to_sql() - DataFrame 写入 MySQL

```
from sqlalchemy import create_engine

# 创建 SQLAlchemy 引擎
engine = create_engine("mysql+pymysql://用户名:密码@主机/数据库名?charset=utf8")

# 写入数据
df.to_sql("表名", engine, if_exists="append", index=False)
```

参数	说明
"表名"	写入哪张表
engine	SQLAlchemy 引擎
if_exists	"append" (追加) / "replace" (覆盖) / "fail" (存在则报错)
index=False	不把 DataFrame 的行号写进数据库

5. SQLAlchemy 连接字符串格式

```
mysql+pymysql://用户名:密码@主机地址/数据库名?charset=utf8
```

示例:

```
engine = create_engine("mysql+pymysql://root:Wuzhewuzhe123@localhost/books")
```

三、完整示例代码

示例1：查询数据

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="密码",
    database="bookstore",
    charset="utf8"
)

cursor = conn.cursor()
cursor.execute("SELECT * FROM books;")
result = cursor.fetchall()

for row in result:
    print(row)

cursor.close()
conn.close()
```

示例2：插入数据

```
import pymysql

conn = pymysql.connect(
```

```
host="localhost",
user="root",
password="密码",
database="bookstore",
charset="utf8"
)

cursor = conn.cursor()

sql = "INSERT INTO books (title, author, price, year) VALUES ('平凡的世界',
cursor.execute(sql)
conn.commit() # 必须提交

print("插入成功! ")

cursor.close()
conn.close()
```

示例3: pandas 读取和写入

```
import pymysql
import pandas as pd
from sqlalchemy import create_engine

# 方式1: 用 pymysql 连接读取
conn = pymysql.connect(
    host="localhost",
    user="root",
    password="密码",
    database="bookstore",
    charset="utf8"
)

df = pd.read_sql("SELECT * FROM books", conn)
print(df)
print("平均价格: ", df["price"].mean())

conn.close()
```

```
# 方式2: 用 SQLAlchemy 写入
engine = create_engine("mysql+pymysql://root:密码@localhost/bookstore?char

data = {
    "title": ["红楼梦", "西游记"],
    "author": ["曹雪芹", "吴承恩"],
    "price": [52, 45],
    "year": [1791, 1592]
}
df_new = pd.DataFrame(data)

df_new.to_sql("books", engine, if_exists="append", index=False)
print("写入成功! ")
```

四、练习总结

练习1：连接数据库并读取数据

✅ 成功用 `pd.read_sql()` 读取 student_system 数据库的 students 表

代码：

```
df = pd.read_sql("SELECT * FROM students", conn)
print(df)
```

练习2：用 pymysql 插入数据

✅ 成功插入3条数据（王五、张二、李六），正确使用了 `conn.commit()`

代码：

```
sql = "INSERT INTO students (name, age, score) VALUES ('王五', 22, 99)"
cursor.execute(sql)
conn.commit()
```

练习3：数据分析

✅ 正确计算平均成绩并找出最高分学生

代码亮点：

```
# 平均成绩（用 f-string 格式化保留2位小数）
print(f"平均成绩: {df['score'].mean():.2f}")

# 最高分学生（用 .loc[] 筛选 + .values[0] 取纯字符串）
print(f"最高分同学: {df.loc[df['score']==df['score'].max(), 'name'].values[0]}")
```

五、常见错误与解决

错误1：pip 无法识别

现象： `pip` : 无法将"`pip`"项识别为 `cmdlet...`

解决：改用 `python -m pip install 包名`

错误2：Unknown database 'xxx'

原因：数据库名写错或数据库不存在

解决：在 MySQL 里运行 `SHOW DATABASES;` 查看实际数据库名

错误3：插入/更新/删除后数据没变化

原因：忘记调用 `conn.commit()`

解决：INSERT / UPDATE / DELETE 后必须加 `conn.commit()`

错误4：print() 里用 `:.2f` 格式化报错

错误写法： `print("平均: ", df["score"].mean():.2f)`

正确写法： `print(f"平均: {df['score'].mean():.2f}")`

原因：格式化符号 `:.2f` 只能在 f-string 的 `{}` 里使用

六、重要知识点

1. Series 和 .values[0]

pandas 筛选返回的是 **Series**（带索引和类型信息），不是纯字符串：

```
df.loc[df['score']==max, 'name']  
# 返回：3      王五  
#      Name: name, dtype: object  
  
df.loc[df['score']==max, 'name'].values[0]  
# 返回：'王五'（纯字符串）
```

2. cursor vs pd.read_sql

	cursor 方式	pd.read_sql 方式
需要游标	✅ 需要 <code>cursor = conn.cursor()</code>	❌ 不需要
返回类型	列表（每行是元组）	DataFrame
后续处理	需要手动遍历	直接用 pandas 操作

3. pymysql vs SQLAlchemy

	pymysql	SQLAlchemy
连接方式	<code>pymysql.connect(host=..., user=...)</code>	<code>create_engine("mysql+pymysql://...")</code>
适用场景	<code>pd.read_sql()</code> 读取	<code>df.to_sql()</code> 写入
特点	参数分开传	所有信息合并成一个连接字符串

七、学生提问与解答

Q1: 为什么 `pd.read_sql()` 不需要 `cursor`?

A: `pd.read_sql()` 内部自动创建和管理 `cursor`，直接把查询结果转成 `DataFrame`，所以我们不需要手动创建 `cursor`。

Q2: `.values[0]` 是什么意思?

A:

- `.values` 把 Series 转成纯数组
- `[0]` 取数组第一个元素
- 作用：把 Series（带索引和类型）转成纯字符串或数字

Q3: 为什么 f-string 里的引号要用单引号？

A: f-string 外层用双引号 `f"..."`，里面的字符串要换成单引号 `'...'`，避免引号冲突。

```
# 错误
print(f"最高分: {df["score"].max()}") # 引号冲突

# 正确
print(f"最高分: {df['score'].max()}")
```

Q4: 为什么 `print("xxx", value:.2f)` 会报错？

A: 格式化符号 `:.2f` 只能在 f-string 的 `{}` 里使用，不能直接用在 `print()` 的逗号分隔参数里。

```
# 错误
print("平均: ", df["score"].mean():.2f)

# 正确
print(f"平均: {df['score'].mean():.2f}")
```

八、下一步学习

Day 9 将学习 **matplotlib 数据可视化**，把 MySQL 数据库和 pandas 的数据用图表展示出来。